S(M,T): Scoring AI Endpoints and the Measurement Gap in LLM Routing

Pranav Lakherwal

pranav@pranavlakherwal.com

March 2026

v1.1. Restructured for clarity and conciseness.
Convergence proofs moved to appendix. Added design reasoning,
self-critique, and Goodhart framing. See CHANGELOG.md for details.
Previous version: `github.com/pranavlakherwal/smt-router/releases/tag/v1.0`

Abstract

Routing AI queries to the right endpoint is becoming critical infrastructure as organizations spend billions on heterogeneous AI systems (LLMs, agents, scripts, tools). However, no existing framework scores these diverse endpoint types through a single equation, and no prior work documents why hand-designed scoring functions fail systematically.

We propose S(M,T), a scoring framework that multiplies three terms: binary gates enforcing hard constraints, a weighted compatibility function, and a task-conditional cost penalty. Unlike existing routers, the gate layer encodes type-specific constraints (scripts have no context window, agents have no token cost), enabling a single equation to route across heterogeneous endpoint pools.

We attempted to compute the compatibility functions from 2.76M public benchmark records. Fourteen designs across four research cycles all failed. The reasons are structural: metrics use incomparable scales, benchmark performance does not transfer to deployment, and complex properties collapse when reduced to single numbers. We call this the measurement gap.

Replacing hand-designed functions with 16 learned bilinear forms (3.15M parameters, trained on 740K routing samples) achieves 83.63% accuracy on RouterBench and AUC 0.8006 on RouteLLM, retaining 94.35% of best-model quality at half the cost. The model generalizes across both benchmarks without benchmark-specific tuning. A 4.6× scaling experiment confirms that data quality, not model capacity, is the binding constraint.

This enables a path toward general-purpose AI routing: one equation that scores any endpoint against any task, where the architect chooses the routing framework that fits their problem. Code: `github.com/pranavlakherwal/smt-router`.

1 Introduction

A user sends a request: "Solve this differential equation step by step and return the answer as LaTeX." You have 13 endpoints: seven LLMs at different price points, three agents, two coding assistants, and a script library. Which one do you call?

This is the routing problem. Get it right and you save money while matching quality. Get it wrong and you either pay for an expensive model that adds nothing, or you pick a cheap one that cannot handle the task.

1.1 Why This Matters

The conversation about AI cost centers on which model to use: frontier versus open source, GPT-4 versus Claude versus Llama. That is the wrong frame. The real leverage is not in the model. It is in the orchestration.

Organizations will spend billions of dollars on AI tokens annually. Most of that spend flows through routing decisions that are manual, heuristic, or nonexistent. Effective routing (sending each task to the right endpoint at the right cost) could redirect billions of dollars of compute. Not cut it. Use it better.

As AI systems grow more heterogeneous (LLMs, agents, scripts, tool-use systems, workflows), routing becomes critical infrastructure. The difference between organizations that get the best outcome per dollar of compute and organizations that overpay for mediocre results is not which model they picked. It is whether they route intelligently.

1.2 The Hypothesis

We approach this with a specific hypothesis: every endpoint (LLM, API, script, agent, workflow) and every task (user prompt, system request, batch job) can be decomposed into measurable properties. If an endpoint's capabilities can be quantified and a task's requirements can be quantified, then the match between them can be scored. If it can be scored, it can be optimized.

This is not obvious. The dominant assumption is that routing requires either human judgment, learned classifiers trained per model pair, or cascading heuristics. We propose that routing is a measurement problem: score every endpoint against every task through a single equation, then pick the highest score.

1.3 What Existing Routers Do

Existing routers have made real progress, but they share assumptions that limit them.

FrugalGPT [Chen et al., 2023] introduced cascading: try the cheap model, escalate if it is not confident. This cuts costs 50 to 90% for simple workloads but assumes models are ordered from weak to strong, which breaks when a cheap coding model outperforms an expensive general model on Python tasks. RouteLLM [Ong et al., 2024] trains pairwise classifiers on Chatbot Arena data. Strong for binary routing. Extending pairwise classifiers to N endpoints requires $O(N^2)$ classifiers. GraphRouter [Feng et al., 2024] uses GNN-learned embeddings over task-model graphs. Dekoninck et al. [2024] unified routing and cascading. RouterBench [Hu et al., 2024] standardized evaluation across 11 methods.

These systems share one assumption: they route between LLMs only. Most use additive or linear scoring (Dekoninck et al.'s $\tau_i = \hat{q} - \lambda \hat{c}$, BEST-Route's cost-quality selection). The learned-representation methods (GraphRouter, ZeroRouter, EmbedLLM) use dot-product scoring, which avoids the additive trap but still lacks hard constraint enforcement: a model that cannot fit the input can still score high. None routes to non-LLM endpoints. Scripts have no context window. Agents have no token cost. MCP tools support specific operations but not arbitrary prompts. Routing across these types requires constraint enforcement that additive and embedding-based scores do not provide.

1.4 What S(M,T) Does

S(M,T) is a single equation that scores any endpoint against any task. Three terms, multiplied:

$$S(M,T) = \underbrace{\left[\prod_j g_j(M,T)\right]}_{\text{Gates}} \cdot \underbrace{\left[\sum_i w_i \cdot \phi_i(M,T)\right]}_{\text{Compatibility}} \cdot \underbrace{e^{-\beta(T)\cdot \mathbf{H}(M)}}_{\text{Cost penalty}}$$

Gates check hard constraints: can this endpoint handle this task at all? Binary pass/fail. If any gate fails, the score is zero. **Compatibility** scores how well the endpoint fits the task across 16 dimensions. **Cost** penalizes expensive endpoints with a task-conditional temperature: a medical query tolerates high cost, a formatting task does not.

The multiplication is the key design choice. These layers are hierarchical, not interchangeable. A failed gate cannot be compensated by high compatibility. Additive scoring allows this. Multiplicative scoring does not.

The techniques are standard. The gate layer is a Product-of-Experts architecture [Hinton, 2002] with structural parallels to hierarchical MoE gating [Jacobs et al., 1991, Jordan and Jacobs, 1994]. The compatibility term is an additive utility function from MCDA [Fishburn, 1967]. The cost penalty is a Boltzmann factor from discrete choice theory [McFadden, 1974, Train, 2009]. Online weight learning uses Robbins-Monro stochastic approximation [Robbins and Monro, 1951]. We do not claim novelty for these components individually. The contribution is the assembled framework and what we found when we tried to implement it.

1.5 Why This Direction Matters

If the hypothesis holds (endpoints and tasks are mathematically decomposable), then routing reduces to measurement. Measure the right properties, and the equation does the rest. Any new model, agent, or tool can be onboarded by measuring its properties and feeding them into the same scoring function. No retraining. No per-pair classifiers. One equation for the entire ecosystem.

We tested this. We designed 16 compatibility functions by hand: reasoning depth, code ability, instruction following, cost efficiency, and twelve others. We tried to compute them from 2.76M public benchmark records. Fourteen designs, four research cycles. All fourteen failed.

The reasons are structural, not fixable with more data: benchmarks use incomparable scales, performance does not transfer from evaluation to deployment, and complex properties lose their task-conditional structure when reduced to single numbers. We call this the *measurement gap*.

But the direction was sound. The equation structure (gates $\times$ compatibility $\times$ cost) was right. The problem was how we computed compatibility. When we replaced the hand-designed functions with 16 learned bilinear forms, the same public data that failed hand-designed measurement achieved 83.63% accuracy on RouterBench and 94.35% quality retention at 50% cost on RouteLLM. The model generalizes across both benchmarks without benchmark-specific tuning.

The measurement gap is the real bottleneck. Not compute. Not model size. Close it, and routing becomes a solved infrastructure problem.

1.6 Contributions

1. **S(M,T)**, a scoring framework that routes across heterogeneous endpoints (LLMs, agents, scripts, tool-use systems) through a single equation. The multiplicative gate structure enables type-specific constraint enforcement. No existing router handles this mix. The techniques are standard; the ap-

plication to heterogeneous routing is new.

2. **The measurement gap**, a systematic documentation showing 0 of 14 hand-designed scoring functions can be reliably computed from public benchmarks, with five identified structural causes.
3. **Learned compatibility functions** that achieve cross-benchmark generalization, with evidence that data quality, not model capacity, is the binding constraint.
4. **The hypothesis** that routing is a measurement problem: if endpoints and tasks are mathematically decomposable, a single equation can score any endpoint against any task. The cross-benchmark results are early evidence that this decomposition is achievable.

2 The Scoring Equation

An endpoint M is any callable capability: an LLM, an agent, a script, or a tool-use system. A task T is a user request with an estimated type, complexity, and context length.

2.1 Design Reasoning

Why these three terms? And why multiply them?

Three things matter when choosing an endpoint for a task, and they are not interchangeable.

Some requirements are non-negotiable. If a model's context window cannot fit the input, nothing else matters. If it does not support structured output and the task requires JSON, it is out. These are constraints, not preferences. We represent them as binary gates. A gate returns 1 (passes) or 0 (fails). The product of all gates is 1 only if every constraint is met.

"Good" is not one dimension. A model that excels at code generation may be poor at creative writing. A model trained for English may struggle with Hindi. Compatibility must be multi-dimensional, weighted by what the task actually requires. The 16 ϕ functions each capture a different aspect of fit. The weights are learned from routing feedback using Robbins-Monro updates (Appendix A), starting from literature-derived seeds.

Cost tolerance belongs to the task, not the model. The same model at the same price might be worth it for a medical analysis and wildly overpriced for reformatting a paragraph. The Boltzmann cost penalty $e^{-\beta(T)\cdot\mathbf{H}(M)}$ adjusts the score based on how cost-sensitive the task is. This is structurally identical to the logit kernel in discrete choice theory [McFadden, 1974]; Train [2009] provides a comprehensive treatment.

Multiplication makes these layers hierarchical. You cannot compensate for a failed gate with excellent compatibility. For routing, some dimensions are dealbreakers, not tradeoffs. This is a Product-of-Experts architecture [Hinton, 2002]: any single gate can veto. If every endpoint fails at least one gate, all scores are zero and the router signals that no endpoint qualifies, triggering fallback (relax the lowest-priority gate and re-score).

2.2 Formal Definition

$$S(M,T) = \underbrace{\left[\prod_{j=1}^{J} g_j(M,T)\right]}_{\text{Gate layer}} \cdot \underbrace{\left[\sum_{i=1}^{K} w_i \cdot \phi_i(M,T)\right]}_{\text{Weighted compatibility}} \cdot \underbrace{e^{-\beta(T)\cdot \mathbf{H}(M)}}_{\text{Cost penalty}} \tag{1}$$

The three terms multiply. If any gate returns 0, the entire score is 0, regardless of compatibility or cost.

2.3 Gates

Each gate $g_j : \mathscr{M} \times \mathscr{T} \to \{0,1\}$ checks a hard constraint. We use five:

1. **Context window**: Can M fit the input? $g_1 = \mathbf{1}[c_M \geq \ell_T]$
2. **Format support**: Does M support structured output, if required?
3. **Availability**: Is M currently reachable?
4. **Budget**: Is the estimated cost within the task's budget?
5. **Thinking support**: Does M support chain-of-thought, if required?

The gate layer encodes type-specific constraints. Scripts have no context window limit but fixed capabilities. Agents have no token cost but variable latency. MCP tools support specific operations but not arbitrary prompts. This heterogeneity is handled through gate design, not separate scoring logic per endpoint type.

2.4 Compatibility Functions

The $K = 16$ phi functions $\phi_i : \mathscr{M} \times \mathscr{T} \to [0,1]$ measure different aspects of how well M fits T. The weight vector $\mathbf{w}$ lies on the probability simplex ($\sum_i w_i = 1$, $w_i \geq 0$).

Table 1. The 16 phi functions with seed weights and data backing.

ID	Name	Type	w_i	Records	Models
ϕ_1	constraint_filter	Computed	0.071	4,200	350
ϕ_2	domain_classifier	Data	0.035	2,042,817	14,926
ϕ_3	complexity_estimator	Data	0.106	2,042,817	14,926
ϕ_4	performance_predictor	Data	0.176	2,759,877	15,176
ϕ_5	context_fitness	Prior	0.008	0	0
ϕ_6	historical_success	Data	0.141	2,307,010	14,928
ϕ_7	uncertainty_score	Derived	0.016	0	0
ϕ_8	cascade_position	Data	0.124	2,311,210	15,278
ϕ_9	load_availability	Runtime	0.008	0	0
ϕ_{10}	cross_node_capability	Prior	0.008	0	0
ϕ_{11}	calibration_quality	Prior	0.008	0	0
ϕ_{12}	output_structure	Prior	0.016	0	0
ϕ_{13}	reasoning_depth	Data	0.106	2,003,949	12,434
ϕ_{14}	cost_efficiency_ratio	Data	0.088	719,702	576
ϕ_{15}	context_window_util	Data	0.018	1,400	350
ϕ_{16}	instruction_adherence	Data	0.071	113,853	5,795

The functions fall into four categories. **Data-grounded** (ϕ_2 through ϕ_4, ϕ_6, ϕ_8, ϕ_{13} through ϕ_{16}): computed from 2.76M normalized benchmark records, carrying 85% of total seed weight. **Computed** (ϕ_1, ϕ_{12}, ϕ_{15}): deterministic functions of metadata. **Prior-based** (ϕ_5, ϕ_{10}, ϕ_{11}): literature-informed defaults,

low weight. **Runtime** (ϕ_7, ϕ_9): measured at inference time.

2.5 Cost Penalty and Weight Learning

The term $e^{-\beta(T)\cdot\mathbf{H}(M)}$ penalizes expensive endpoints with a task-conditional temperature. A safety-critical medical query gets a low β (accept high cost); a formatting task gets a high β (prefer cheap).

Initial weights are derived from routing literature [Chen et al., 2023, Ong et al., 2024, Hu et al., 2024] and data quality scores (Equation 5 in Appendix A). A decaying entropy bonus inspired by Soft Actor-Critic [Haarnoja et al., 2018] prevents mode collapse during early routing. The online weight learner adjusts weights from routing feedback using Robbins-Monro stochastic approximation [Robbins and Monro, 1951]. Full details on seed weight derivation, entropy scheduling, and update rules are in Appendix A.

3 The Measurement Gap

With the equation defined, we set out to implement it. We had 2.76M benchmark records from seven public datasets spanning 15,526 models.

Table 2. Public datasets ingested for phi function grounding.

Dataset	Records	Models	Metric Type
Open LLM Leaderboard	1,897,419	5,168	Accuracy
RouteLLM	714,606	2	Win rate
RouterEval	105,042	7,117	Pairwise preference
Epoch AI	40,356	2,641	Mixed
OpenRouter	4,200	350	Pricing/context
SWE-bench	1,488	149	Pass rate
AlpacaEval	966	102	Win rate
Total	**2,764,077**	**15,526**	

For each of the 16 phi functions, we wrote an algorithm to compute a $[0, 1]$ score from these records. We tested 14 designs across four research cycles. Our verification process ran overnight: a mathematical verification engine (using DeepSeek Prover V2 and DeepSeek R1) stress-tested each design for structural flaws, adversarial inputs, and consistency with published results.

All fourteen designs failed. Not "performed poorly." Failed. Each one contained at least one structural flaw that made it unreliable for routing.

3.1 Walking Through Specific Failures

Cost efficiency (ϕ_{14}). The idea is simple: divide benchmark score by price per token. Higher is better. But benchmark score is in units of accuracy (or Elo, or pass rate) and price is in dollars per token. The ratio is dimensionally invalid. Worse, a 10-token formatting request and a 4,000-token analysis get the same benchmark score but wildly different costs. Task-length sensitivity causes prediction errors exceeding $100\times$.

Reasoning depth (ϕ_{13}). We tried to estimate reasoning complexity from prompt features: number of sub-questions, logical operators, domain keywords. Surface features correlate poorly with actual reasoning depth. And GSM8K performance (our best data source for reasoning) does not generalize to non-mathematical reasoning. A model that solves grade-school math is not necessarily good at legal

analysis.

Instruction adherence (ϕ_{16}). IFEval measures constraint satisfaction (did the model follow the format?). MT-Bench measures stylistic quality (was the response good?). These are different skills. Averaging them predicts neither. A model that scores high on IFEval by rigidly following templates may score low on MT-Bench for being robotic. We cannot add these signals without destroying what each one measures.

3.2 Root Causes

Every failure traces to five structural problems, each a variant of Goodhart's Law [Goodhart, 1975]: when a measure becomes a target, it ceases to be a good measure. Campbell [Campbell, 1979] documented the same pattern in social science indicators. Manheim and Garrabrant [Manheim and Garrabrant, 2019] taxonomize four variants; we observe three in our data.

1. **Metric incomparability (Regressional Goodhart).** Benchmarks use fundamentally different scales (accuracy, Elo, pass@k, win rate). Min-max normalization creates comparable numbers but destroys semantic content. The normalization itself introduces systematic bias.

2. **Domain transfer breakdown (Extremal Goodhart).** Performance on curated benchmark tasks does not predict performance on real user tasks. Models optimized for benchmarks perform unpredictably on out-of-distribution routing tasks. Evaluation conditions differ systematically from deployment conditions.

3. **Dimensional collapse.** Reducing complex phenomena (reasoning depth, instruction adherence, cost efficiency) to single scalars destroys the task-conditional structure that routing needs. A model's reasoning ability depends on the domain, the depth required, and the output format. One number cannot capture this.

4. **Benchmark contamination (Adversarial Goodhart).** LLMs are trained on benchmark test sets, inflating scores non-uniformly [Deng et al., 2023]. Normalization cannot remove this systematic bias. Some models appear stronger than they are on specific tasks.

5. **Sparse measurement.** Benchmarks provide 5 to 7 data points for phenomena that need dozens. Interpolation fails at exactly the operating points routing depends on.

3.3 The Pivot

The measurement gap applies specifically to hand-designed, interpretable phi functions that require explicit metric normalization and domain mapping. It does not mean S(M,T) is wrong.

The gate structure works: binary pass/fail checks on context window, format support, availability, budget, and thinking capability are straightforward to compute and do not depend on benchmarks. The cost penalty works: pricing data is clean and available. The problem is specifically in the compatibility functions, where hand-designed measurement requires mapping benchmark metrics onto routing-relevant scores.

The equation was right. The implementation method was wrong. So we kept the equation and changed the method.

4 Learned Compatibility Functions

Hand-designed phi functions fail because they require explicit metric normalization and domain mapping (Section 3). But routing does not need interpretable phi functions. It needs phi functions that predict routing outcomes. This section replaces the 16 semantic phis with 16 learned bilinear forms trained end-to-end.

4.1 Architecture

Each learned phi is a bilinear interaction between a prompt embedding and a model embedding:

$$\hat{\phi}_i(M,T) = \sigma\left(\mathbf{e}_T^\top \mathbf{W}_i \mathbf{e}_M + b_i\right), \quad i = 1,\ldots,K \tag{2}$$

where $\mathbf{e}_T \in \mathbb{R}^{768}$ is a frozen prompt embedding (all-mpnet-base-v2), $\mathbf{e}_M \in \mathbb{R}^{256}$ is a learned model embedding, $\mathbf{W}_i \in \mathbb{R}^{768\times 256}$ is a learned bilinear matrix, and σ is the sigmoid. The overall score is:

$$\hat{S}(M,T) = \sum_{i=1}^{K} \alpha_i \cdot \hat{\phi}_i(M,T), \quad \alpha_i = \frac{e^{a_i}}{\sum_j e^{a_j}} \tag{3}$$

This is a multi-head factorization machine [Rendle, 2010] with sigmoid outputs, building on the matrix factorization foundations established by Koren et al. [2009] (the same paradigm underlying RouteLLM's matrix factorization router). Multi-head attention [Vaswani et al., 2017] provides the structural inspiration for multiple heads. Our contribution is applying this to routing and demonstrating that it resolves the measurement gap.

With $K = 16$ heads, the model has 3.15M trainable parameters: 16 bilinear matrices (3.15M), 48 model embeddings (12K), 16 head weights, and 16 biases. A diversity regularization term $\mathscr{L}_{\text{div}} = \frac{1}{K(K-1)} \sum_{i\neq j} |\cos(\text{vec}(\mathbf{W}_i), \text{vec}(\mathbf{W}_j))|$ prevents head collapse.

4.2 Training Data

Five public datasets, unified into 740,803 (prompt, model, score) triples:

Table 3. Training data sources for the BilinearPhiEngine.

Source	Samples	Models	Score Type
RouterBench	401,467	11	Accuracy (0/1)
RouteLLM	218,202	2	Win rate (0/1)
Arena 55K	111,712	45	Elo-derived
MT-Bench	6,710	5	Rating (1 to 10)
RewardBench	2,712	31	Accuracy (0/1)
Total	**740,803**	**48**	

A canonical registry maps 180 model name aliases to 48 canonical IDs. Scores are normalized to $[0,1]$ per source. Data is split 80/10/10. Training uses AdamW with differential learning rates (10^{-4} for bilinear matrices, 10^{-3} for embeddings) and early stopping. Total wall time: under 70 minutes on Apple Silicon.

4.3 Implementation

The v0.1 router implements S(M,T) across 13 heterogeneous endpoints: 7 LLMs (Claude Opus 4, Claude Sonnet 4, GPT-4o, DeepSeek R1, DeepSeek V3, Qwen3 Coder, Gemini 2.5 Pro), 3 agents (search, math verification, first-principles), 1 script (session capture), and 2 MCP tool-use systems (Notion, Google Workspace). The gate layer handles type-specific constraints (agents have no context window limit, scripts have fixed capabilities, MCP tools support specific operations), and phi functions score all endpoint types through a common interface.

4.4 Results

4.4.1 RouterBench

RouterBench [Hu et al., 2024] tests multi-model selection across 11 models and 8 datasets using accuracy as the metric.

Table 4. RouterBench results. Our router approaches always-strong while using cheaper models.

Method	0-shot	5-shot	Overall
Random	55.50	64.34	–
Always-strong	84.35	86.66	85.51
BilinearPhi (ours)	**80.49**	**86.76**	**83.63**
Oracle	96.13	96.76	96.45

In the 5-shot setting, the router slightly beats always-strong (86.76% vs. 86.66%). This means it learns when cheaper models are genuinely better, not just cheaper. The gap to always-strong is 1.88 percentage points. The gap to oracle is 12.82 points, representing the upper bound on what any router could gain.

4.4.2 RouteLLM

RouteLLM [Ong et al., 2024] evaluates binary routing across MMLU, GSM8K, and MT-Bench.

Table 5. RouteLLM results. At 50% strong-model cost, 94.35% of quality is retained.

Threshold	Quality	Quality Ratio
Always weak	0.7187	84.09%
50% strong	0.8065	94.35%
Always strong	0.8547	100.00%
AUC	**0.8006**	

In practice: $10,000/month on AI becomes $5,000 with 94% quality. The quality drop concentrates on queries near the decision boundary where both models perform similarly. The model was not trained for RouteLLM's binary task specifically. AUC 0.8006 reflects general routing ability, not benchmark-specific optimization.

4.5 Cross-Benchmark Generalization

The BilinearPhiEngine was not trained for either benchmark specifically. It learned from five data sources simultaneously and generalized to both evaluation paradigms, which use different metrics (accuracy vs. quality retention), different model sets (11 models vs. 2), and different task distributions.

Three findings. **RouterBench is the anchor**: removing it collapses cross-source generalization (row 1 drops to $r < 0.20$ on non-RouterBench sources). **RouteLLM resists transfer**: capped at $r \approx 0.32$ regardless of training composition, because its 2-model binary format is a qualitatively different problem.

Table 6. Leave-one-out generalization. Diagonal (bold) = zero-shot transfer to the held-out source.

Trained w/o	RBench	RLLM	Arena	MTB	RwdB
RouterBench	**0.651**	0.073	0.197	0.174	0.169
RouteLLM	0.425	**0.319**	0.457	0.468	0.385
Arena 55K	0.583	0.321	**0.460**	0.470	0.386
MT-Bench	0.682	0.321	0.459	**0.469**	0.385
RewardBench	0.679	0.320	0.459	0.468	**0.385**

Small sources are redundant: removing MT-Bench or RewardBench has no measurable effect on generalization.

This is early evidence that a general-purpose routing framework is achievable. The learned scoring functions appear to capture task-model compatibility at a level more general than any single benchmark measures. We do not claim this is proven. But the cross-benchmark results, combined with the leave-one-out stability, point in this direction.

5 Scaling Does Not Help

A natural question: does scaling the BilinearPhiEngine improve routing? We built V3 to test this. The answer is no.

V3 design. V3 scales three dimensions simultaneously: $K = 24$ heads with $D = 768$ (14.6M parameters, 4.63× V1), training on 1.98M rows from 10 additional preference datasets, and three architectural additions (task conditioning, cross-attention, metadata features) via 5-stage progressive training on an A100.

Results. V3 underperforms V1 on every benchmark:

Table 7. V1 vs. V3 on external benchmarks. V1 wins everywhere.

Model	RouterBench	RouteLLM AUC	Q@50%
V1 (3.15M)	**83.63**	**0.8006**	**0.9435**
V3-Stage2 (14.6M)	82.94	0.7972	0.8020
V3-Stage5 (14.6M)	81.47	0.7973	0.8022

The quality-at-50%-strong metric shows the starkest gap: V3 retains only 80% of best-model quality at 50% cost, versus V1's 94%. On internal evaluation, V3's correlation with ground truth collapses from $r = 0.53$ (V1) to $r \approx -0.04$ (V3).

Why scaling failed. Four factors: (1) **Label distribution shift**: the V2 training data merged preference-based scores with benchmark accuracy scores, encoding different quality semantics. (2) **Sparse model coverage**: 53 of 101 model IDs were unrecognized, causing 49% of rows to be skipped. (3) **Progressive training instability**: Stage 5 performs worse than Stage 2, confirming architectural additions overfit to training artifacts. (4) **Overparameterization**: V3's parameter-to-sample ratio (7.4:1) exceeds V1's (4.3:1), absorbing noise rather than learning better representations.

Measurement gap confirmation. This result confirms the measurement gap thesis from a different angle. If the bottleneck were model capacity, V3 should outperform V1. It does not. The bottleneck is

in the training signal: inconsistent scoring semantics across data sources, label noise from aggregating heterogeneous evaluation methodologies, and the absence of routing-specific supervision. The path to better routing runs through cleaner data, not bigger models.

6 Theoretical Properties

S(M,T) satisfies standard convergence conditions. The phi functions are bounded in $[0,1]$ and step sizes decay as $c/(t+1)$, so weight convergence follows from Robbins-Monro [Robbins and Monro, 1951]. Regret bounds of $O(\sqrt{KN\log N})$ follow from Thompson Sampling for contextual bandits [Agrawal and Goyal, 2013]. The techniques are standard; the contribution is verifying that S(M,T) meets their conditions. Full proofs, including identifiability results and the gate correctness property, are in Appendix A.

7 Related Work

Routing Architectures. FrugalGPT [Chen et al., 2023] cascades from cheap to expensive models. RouteLLM [Ong et al., 2024] trains pairwise classifiers for binary routing. RouterBench [Hu et al., 2024] standardized evaluation. Hybrid LLM [Ding et al., 2024] routes by difficulty. AutoMix [Madaan et al., 2023] uses self-verification for escalation. Dekoninck et al. [2024] unified routing and cascading. BEST-Route [Ding et al., 2025] jointly selects models and response counts. MoMA [Li et al., 2025] and DAAO [Zhou et al., 2025] extend to LLM-backed agents but not to deterministic scripts or non-AI tools. Each contribution is real. The shared limitation: LLM-to-LLM routing with continuous scoring.

S(M,T) differs structurally. The gate layer enforces hard constraints through multiplication [Fishburn, 1967, Train, 2009]: failing any gate produces zero. The framework routes across heterogeneous endpoints (7 LLMs, 3 agents, 1 script, 2 MCP tool-use systems) through one equation. No existing framework handles this mix.

Learned Routing Representations. RouterDC [Chen et al., 2024] trains dual contrastive embeddings. EmbedLLM [Zhuang et al., 2024] learns compact model vectors for performance prediction. ICL-Router [Wang et al., 2025] uses in-context vectors. Prompt-to-Leaderboard [Frick et al., 2025] predicts Bradley-Terry coefficients from prompts. ZeroRouter [Yan et al., 2026] creates model-agnostic latent spaces for zero-shot onboarding.

GraphRouter [Feng et al., 2024] is the closest architectural comparison. Both systems learn interaction representations between tasks and models rather than hand-designing scoring rules. The differences are structural. GraphRouter's reward labeling uses a weighted additive formula ($\alpha \cdot \text{Performance} - \beta \cdot \text{Cost}$), meaning a model that fails a hard constraint can still receive a positive reward if its performance score is high enough. S(M,T) enforces hard constraints through multiplicative gates: failing any gate produces zero, regardless of compatibility or cost. GraphRouter also assumes all endpoints are LLMs, while S(M,T)'s gate layer encodes type-specific constraints for heterogeneous endpoints. GraphRouter's GNN architecture captures richer relational structure between tasks and models; S(M,T)'s bilinear forms are simpler but integrate into a framework with explicit constraint enforcement and task-conditional cost.

Our BilinearPhiEngine is in this learned-representation family. The architecture (multi-head bilinear forms) is structurally a factorization machine [Rendle, 2010], a well-established approach in recommendation systems. What differs is the integration: the learned forms serve as drop-in replacements for the hand-designed phi functions within the S(M,T) framework. The gate layer, cost penalty, and weight learning structure remain unchanged. This modularity is by design: the measurement gap motivates re-

placing the compatibility functions specifically, while keeping constraint enforcement and cost structure intact.

Online Learning. Thompson Sampling for contextual bandits [Agrawal and Goyal, 2013] provides the regret framework. BaRP [Hao et al., 2025] models routing as multi-objective contextual bandits. OmniRouter [Mei et al., 2025] uses Lagrangian dual decomposition under budget constraints. We combine Robbins-Monro weight updates [Robbins and Monro, 1951] with SAC-style entropy decay [Haarnoja et al., 2018].

8 Discussion

8.1 What the Results Say

The learned scoring functions work but sacrifice interpretability. The 16 bilinear heads carry no semantic labels. Whether this tradeoff is acceptable depends on the use case. For production systems that need auditability, hand-designed functions with online weight learning remain appropriate. What matters for the field is that the cross-benchmark generalization (Section 4) suggests general-purpose routing is achievable. We use this router in production. This is incomplete research, and we want to be explicit about what is next.

8.2 Self-Critique and Honest Assessment

We ran systematic falsification against every novelty claim using automated verification (DeepSeek Prover V2, DeepSeek R1). The results shaped this paper's posture.

S(M,T) maps to a constrained conditional logit model [McFadden, 1974]. The bilinear forms are factorization machines [Rendle, 2010]. The gates inherit from hierarchical Mixture-of-Experts gating [Jordan and Jacobs, 1994]. The convergence proofs use standard Robbins-Monro and Thompson Sampling results. We do not claim these techniques are novel.

We also acknowledge visible gaps. The 13-endpoint pool is small. We have no live A/B test results. The learned phi functions are uninterpretable. Single-turn scoring misses conversational dynamics. These are real limitations, not footnotes.

8.3 Limitations

Seed weight subjectivity. Literature prior scores were assigned by a single researcher. Different reviewers might shift the seed vector. The online learner corrects for this over time, but early routing decisions depend on the prior.

Small endpoint set. The router covers 13 endpoints. Production systems may need hundreds. Gate evaluation cost scales linearly with endpoint count.

No live A/B testing. We have not compared S(M,T) against human routing or competing algorithms on live traffic.

Single-turn only. The framework scores each task independently. Multi-turn conversations, where the optimal endpoint may change mid-conversation, are not addressed.

8.4 Future Directions

The equation can get better. This is step one, not the final form. The future does not belong to any single routing framework. It belongs to the decision each architect makes: which framework fits their problem, their endpoints, their constraints. Our goal is to make S(M,T) one strong option in that decision space.

Expanding the scoring functions. 16 learned phi dimensions is a starting point. We are actively testing additional functions across broader task distributions. The modular design makes this straightforward: learned phi functions are drop-in replacements within the gates-compatibility-cost structure.

Shadow routing (coming soon). The biggest barrier to better routing is the absence of routing-specific training data. We are developing an open-source shadow router: it monitors queries and context, autonomously calculates which endpoint it would assign at the backend, and logs the result without interfering with the existing workflow. Think of it as a clinical trial for AI usage: observe, measure, compare, then decide. With enough data, this generates real routing signal that benchmarks cannot provide. Details: `b@bnotb.com`.

Richer cost modeling. Price per token is insufficient. Production cost includes latency, rate limits, reliability, and multi-step overhead.

Multi-turn routing. Extending S(M,T) to conversations requires a state variable: $S(M, T, s_t)$ where s_t tracks accumulated context, cost, and quality across turns.

Data curation over model scaling. The V3 result (Section 5) confirms the bottleneck is data quality, not model capacity. Future improvements should focus on consistent scoring semantics across training sources.

8.5 Robustness

The multiplicative structure provides three built-in defenses. Entropy regularization prevents mode collapse. Task-conditional temperature stabilizes rankings under noisy phi values. Decaying step sizes in the weight learner, combined with simplex projection, limit the impact of adversarial feedback. The gate layer is not affected by weight learning: manipulated weights cannot override hard constraints. Full adversarial analysis is in Appendix B.

9 Conclusion

S(M,T) assembles well-understood components (Product-of-Experts gates, additive utility, Boltzmann cost penalty, Robbins-Monro learning) into a scoring framework for routing across heterogeneous AI endpoints. The techniques are standard. The finding is not.

Fourteen attempts to compute hand-designed scoring functions from 2.76M benchmark records all failed. The reasons are structural: metrics use incomparable scales, benchmark performance does not transfer to deployment, and complex properties collapse when reduced to single numbers. This is the measurement gap.

Replacing hand-designed functions with 16 learned bilinear forms achieves 83.63% accuracy on RouterBench and 94.35% of best-model quality at half the cost on RouteLLM. The model generalizes across

both benchmarks without benchmark-specific tuning. A 4.6× scaling experiment confirmed that data quality, not model capacity, is the binding constraint.

The measurement gap separates interpretable from learned routing, but it also points forward. If benchmarks cannot ground hand-designed routing scores, the field needs benchmarks designed for routing, not repurposed from model comparison. Close the measurement gap, and you do not just get better routing. You get a foundation for evaluating AI systems under the conditions where they actually operate.

Code: `github.com/pranavlakherwal/smt-router`

AI Disclosure. AI tools from Anthropic assisted with writing and code development. All research direction, experimental design, and intellectual contributions are the author's own.

References

Shipra Agrawal and Navin Goyal. Thompson sampling for contextual bandits with linear payoffs. In *Proceedings of the 30th International Conference on Machine Learning*, pages 127–135, 2013.

Donald T Campbell. Assessing the impact of planned social change. *Evaluation and Program Planning*, 2(1):67–90, 1979. doi: 10.1016/0149-7189(79)90048-X.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Shuhao Chen, Weisen Jiang, Baijiong Lin, et al. RouterDC: Query-based router by dual contrastive learning for assembling large language models. *Advances in Neural Information Processing Systems*, 37, 2024.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A unified approach to routing and cascading for LLMs. *arXiv preprint arXiv:2410.10347*, 2024. ICML 2025.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerber, and Arman Cohan. Investigating data contamination in modern benchmarks for large language models. *arXiv preprint arXiv:2311.09783*, 2023.

Dujian Ding, Ankur Mallick, Chi Wang, et al. Hybrid LLM: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*, 2024. ICLR 2024.

Dujian Ding, Ankur Mallick, Shaokun Zhang, et al. BEST-Route: Adaptive LLM routing with test-time optimal compute. *arXiv preprint arXiv:2506.22716*, 2025. ICML 2025.

John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th International Conference on Machine Learning*, pages 272–279, 2008.

Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A graph-based router for LLM selections. *arXiv preprint arXiv:2410.03834*, 2024. ICLR 2025.

Peter C Fishburn. Additive utilities with incomplete product set: Applications to priorities and assignments. *Operations Research*, 15(3):537–542, 1967. doi: 10.1287/opre.15.3.537.

Evan Frick, Connor Chen, Joseph Tennyson, Tianle Li, et al. Prompt-to-leaderboard. *arXiv preprint arXiv:2502.14855*, 2025.

Charles A E Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary*

Economics. Reserve Bank of Australia, 1975.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

Bruce Hajek. Cooling schedules for optimal annealing. *Mathematics of Operations Research*, 13(2): 311–329, 1988.

Botao Hao et al. Learning to route LLMs from bandit feedback: One policy, many trade-offs. *arXiv preprint arXiv:2510.07429*, 2025.

Geoffrey E Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002.

Qitian Hu et al. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.

Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994. doi: 10.1162/neco.1994.6.2.181.

Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. doi: 10.1109/MC.2009.263.

Harold J Kushner and G George Yin. *Stochastic Approximation and Recursive Algorithms and Applications*. Springer, 2nd edition, 2003.

Zixuan Li et al. Towards generalized routing: Model and agent orchestration for adaptive and efficient inference. *arXiv preprint arXiv:2509.07571*, 2025.

Aman Madaan, Pranjal Aggarwal, Prakhar Anand, Srividya Pranavi Potluri, Swaroop Mishra Gupta, Dheeraj Rajagopal, Vladlen Koltun, and Yiming Yang. AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2023.

David Manheim and Scott Garrabrant. Categorizing variants of Goodhart's law. *arXiv preprint arXiv:1803.04585*, 2019.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. pages 105–142, 1974.

Kai Mei, Wujiang Xu, Minghao Guo, et al. OmniRouter: Budget and performance controllable multi-LLM routing. *arXiv preprint arXiv:2502.20576*, 2025.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

Steffen Rendle. Factorization machines. In *Proceedings of the 10th IEEE International Conference on Data Mining*, pages 995–1000, 2010. doi: 10.1109/ICDM.2010.127.

Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

Thomas J Rothenberg. Identification in parametric models. *Econometrica*, 39(3):577–591, 1971.

Kenneth E Train. *Discrete Choice Methods with Simulation*. Cambridge University Press, 2nd edition, 2009.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Chenxu Wang, Hao Li, Yiqun Zhang, et al. ICL-Router: In-context learned model representations for LLM routing. *arXiv preprint arXiv:2510.09719*, 2025. AAAI 2026.

Cheng Yan et al. Breaking model lock-in: Cost-efficient zero-shot LLM routing via a universal latent space. *arXiv preprint arXiv:2601.06220*, 2026.

Chen Zhou et al. Difficulty-aware agentic orchestration for query-specific multi-agent workflows. *arXiv preprint arXiv:2509.11079*, 2025.

Richard Zhuang, Tianhao Wu, Zhaojin Wen, et al. EmbedLLM: Learning compact representations of large language models. *arXiv preprint arXiv:2410.02223*, 2024. ICLR 2025.

A Convergence Proofs and Formal Properties

This appendix contains the full convergence analysis, identifiability proofs, and supporting formal results referenced in the main text.

A.1 Weight Learning

After each routing decision n, the weight vector updates via Robbins-Monro stochastic approximation [Robbins and Monro, 1951]:

$$w_i^{(n+1)} = \Pi_{\Delta^{K-1}}\left[w_i^{(n)} + \alpha_n \cdot \phi_i(M_n, T_n) \cdot (r_n - \hat{r}_n)\right] \tag{4}$$

where $\alpha_n = 1/(n+1)$, $\hat{r}_n = \sum_i w_i^{(n)} \phi_i(M_n, T_n)$ is the predicted score, r_n is the observed quality, and $\Pi_{\Delta^{K-1}}$ projects onto the probability simplex [Duchi et al., 2008].

A.2 Seed Weights

The initial weight vector is not arbitrary:

$$w_i^{(0)} = \frac{p_i \cdot q_i}{\sum_{k=1}^{K} p_k \cdot q_k} \tag{5}$$

where p_i is a literature-derived importance score (surveying FrugalGPT [Chen et al., 2023], RouteLLM [Ong et al., 2024], RouterBench [Hu et al., 2024]) and q_i is a data quality multiplier (1.0 for high coverage, 0.1 for no data). This puts the most weight on functions that are both important in the literature and well-supported by data. The online weight learner adjusts these from routing feedback.

A.3 Entropy Regularization

To prevent the router from collapsing to always picking one endpoint, we add a decaying entropy bonus inspired by Soft Actor-Critic [Haarnoja et al., 2018]:

$$S(M,T)_{\text{reg}} = S(M,T) + \lambda(n) \cdot H(\pi), \quad \lambda(n) = \max(\lambda_{\min}, \lambda_0 \cdot \gamma^n) \tag{6}$$

with $\lambda_0 = 0.5$, $\gamma = 0.995$, $\lambda_{\min} = 0.01$. This keeps exploration high early on and converges to near-greedy selection as the router gains confidence.

A.4 Global Convergence

Theorem A.1 (Global Convergence)**.** *With logarithmic cooling $\beta(t) = c \cdot \log(1+t)$, bounded phi functions $\phi_i \in [0,1]$, and step sizes satisfying the Robbins-Monro conditions ($\sum \alpha_n = \infty$, $\sum \alpha_n^2 < \infty$), the weight vector converges almost surely to $\mathbf{w}^* = \arg\max_{\mathbf{w} \in \Delta^{K-1}} \mathbb{E}[\textit{quality}(\arg\max_M S(M,T))]$. This follows from the Hajek sufficient conditions for simulated annealing [Hajek, 1988] combined with the ODE method for stochastic approximation [Kushner and Yin, 2003].*

A.5 Regret Bound

Routing maps to contextual bandits: the router observes context (task T), picks an arm (endpoint M), and gets a reward (quality r). The phi values form the feature vector.

Theorem A.2 (Sublinear Regret)**.** *With $K = 16$ phi functions and L endpoints, Thompson Sampling with linear payoffs [Agrawal and Goyal, 2013] gives cumulative regret:*

$$R(N) = O\left(\sqrt{KN \log N}\right) \tag{7}$$

For $K = 16$ and $L = 13$, reaching 95% optimal routing requires roughly $N \approx 10{,}000$ decisions.

The $\sqrt{K}$ factor (not L) reflects the dimension of the weight space. The contextual structure provides an improvement of $\sqrt{K/L}$ over naive per-endpoint exploration.

A.6 Identifiability

Theorem A.3 (Weight Identifiability)**.** *The weight vector $\mathbf{w}$ is uniquely recoverable from observed routing outcomes if the phi matrix $\Phi \in \mathbb{R}^{N \times K}$ (rows are phi evaluations on observed model-task pairs) has full column rank. With 15,526 models in the database and $K = 16$, this condition is easily met.*

Theorem A.4 (Temperature Identifiability)**.** *Given sufficient task diversity (task feature matrix has full rank), the temperature parameters $\beta(T)$ are jointly identifiable with the weights. Task-conditional temperature improves identifiability over a scalar β because different task types provide independent constraining equations [Rothenberg, 1971].*

A.7 Gate Layer Correctness

Proposition A.5 (PoE Eliminates Equal-Scoring Vulnerability)**.** *If endpoint M_a fails any gate ($g_j(M_a,T) = 0$), then $S(M_a,T) = 0$ regardless of phi values. Under additive scoring, M_a could still outscore a constraint-satisfying endpoint by excelling on other dimensions. The multiplicative gate structure makes this impossible.*

A.8 Informed Prior and Entropy

Proposition A.6 (Prior Acceleration)**.** *Starting from the literature-informed seed weights $\mathbf{w}_0$ (Equation 5) reduces early regret by $\Omega(\|\mathbf{w}_U - \mathbf{w}^*\|^2 - \|\mathbf{w}_0 - \mathbf{w}^*\|^2)$ compared to uniform initialization $\mathbf{w}_U = (1/K, \ldots, 1/K)$, provided the prior is closer to optimal than uniform.*

Proposition A.7 (Entropy Schedule)**.** *The decay* $\lambda(n) = \max(0.01, 0.5 \cdot 0.995^n)$ *ensures exploration dominates for the first* $\sim$100 *decisions (*$\lambda > 0.3$*) and converges to near-greedy selection (*$\lambda \to 0.01$*) without breaking the convergence guarantee of Theorem A.1.*

A.9 Phi Function Independence

Theorem A.8 (Pairwise Orthogonality)**.** *The 16 phi functions capture non-redundant signals. We verify this for the six most confusable pairs by constructing task-model examples where their rankings diverge: calibration vs. uncertainty (model-internal vs. router-external), structure vs. constraint (continuous reliability vs. binary check), reasoning vs. complexity (multi-step depth vs. surface difficulty), cost efficiency vs. cost penalty (task-conditional vs. global), context utilization vs. context gate (degradation curve vs. binary fit), and instruction adherence vs. context fitness (behavioral compliance vs. content relevance). Full separating examples are available in the supplementary materials.*

B Adversarial Analysis

We analyze three attack vectors against the S(M,T) routing framework and show that existing components provide built-in mitigations.

B.1 Mode Collapse

Attack. Without regularization, the router converges to always selecting a single endpoint, ignoring potentially better alternatives for specific task types.

Mitigation. The entropy regularization term (Equation 6) directly addresses mode collapse. With $\lambda_0 = 0.5$, the entropy bonus during early routing is comparable in magnitude to score differences between endpoints. For 13 endpoints, $\max H(\pi) = \log 13 \approx 2.56$, so the initial entropy bonus can reach $0.5 \times 2.56 = 1.28$, which dominates typical score separations of 0.1 to 0.3.

As $\lambda(n)$ decays toward $\lambda_{\min} = 0.01$, the router gradually transitions from exploration to exploitation. Proposition A.7 establishes that this decay is fast enough to converge but slow enough to prevent premature commitment. The minimum entropy floor $\lambda_{\min} > 0$ ensures the router never fully degenerates to deterministic selection, maintaining a small probability of exploring alternatives indefinitely.

B.2 Ranking Instability under Perturbation

Attack. Small changes in phi function outputs cause large changes in endpoint rankings, making the router unreliable. This is particularly concerning when phi functions are noisy (as documented in Section 3).

Mitigation. The task-conditional temperature vector $\beta(T)$ provides stability through two mechanisms:

1. **Temperature smoothing.** The Boltzmann cost penalty $e^{-\beta(T)\cdot \mathbf{H}(M)}$ acts as a soft regularizer. For endpoints with similar phi scores, the temperature term breaks ties based on cost, which is a stable, well-measured quantity (unlike benchmark-derived phi values).

2. **Task-conditional sensitivity.** Different task types tolerate different levels of score perturbation. A safety-critical medical query should have a high β (strong cost tolerance, selecting the best model regardless of price), while a simple formatting task should have a low β (preferring cheaper endpoints). This task-conditional structure means ranking instability in one task type does not propagate to others.

The Product-of-Experts gate layer (Proposition A.5) provides additional stability: endpoints that fail hard constraints are eliminated before scoring, removing the most volatile rankings.

B.3 Weight Manipulation via Feedback

Attack. An adversary submits systematically biased feedback signals to steer the Robbins-Monro weight updates toward a preferred endpoint.

Mitigation. Three properties of the learning system limit this attack:

1. **Decaying step size.** The step size $\alpha_n = 1/(n+1)$ means early feedback has outsized influence, but late feedback has diminishing effect. After 1,000 decisions, each new feedback signal shifts weights by at most 0.1%. An adversary arriving late faces prohibitive cost to overcome accumulated honest feedback.

2. **Simplex constraint.** The Duchi projection onto Δ^{K-1} prevents any single weight from being driven to extreme values. Inflating one weight necessarily deflates others, limiting the damage from biased feedback on a single phi function.

3. **Gate layer independence.** The binary gates are not affected by weight learning. Even if an adversary successfully manipulates weights to favor a particular model, the gate layer still eliminates that model for tasks where it fails hard constraints. The manipulation cannot override context window limits, format support, or budget constraints.

B.4 Summary

Table 8. Attack vectors and built-in mitigations.

Attack	Defense	Component
Mode collapse	Entropy regularization	Equation 6
Ranking instability	Task-conditional $\beta(T)$ + gates	Equation 1
Weight manipulation	Decaying α_n + simplex + gates	Equation 4

These mitigations are not post-hoc patches. They emerge from the framework's multiplicative structure: gates provide hard safety guarantees that soft scoring cannot override, entropy regularization is a standard technique from reinforcement learning [Haarnoja et al., 2018], and decaying step sizes are inherent to Robbins-Monro convergence [Robbins and Monro, 1951].